

# SDA-Pro

---

Security Incident Response & Threat Mitigation Platform

Software Design & Architecture — Semester Project

A Starter Guide for Semester Project

Table of Contents

Part I: Project Charter — Scope, Patterns, Milestones, Rubric

Part II: Repository Guide — Structure, GitFlow, API Contracts, Testing

Part III: UML Class Diagram — All 12 Design Patterns

Part IV: UML Component Diagram — SOA Architecture

Part V: UML Sequence — Alert Ingestion & Enrichment Pipeline

Part VI: UML Sequence — Incident Response Orchestration



# Part I: Project Charter

---

## SDA-Pro — Security Incident Response & Threat Mitigation Platform

---

### Software Design & Architecture — Semester Project Charter

---

#### 1. Executive Summary

**Project Name:** SDA-Pro — Intelligent Security Operations Center (SOC) Platform

**Team Size:** 3 Students

**Duration:** 2 Weeks

**Goal:** Design and implement a distributed security operations platform that ingests threat alerts from multiple sources, enriches and classifies them through an intelligent pipeline, orchestrates automated incident response actions, and provides real-time situational awareness to SOC analysts — demonstrating mastery of all 12 design patterns, SOA, MVC, Layered, and Event-Driven architectures.

**Domain Context:** Modern enterprises face thousands of security alerts daily from firewalls, EDR tools, SIEMs, and cloud platforms. SDA-Pro automates the triage and response pipeline, reducing mean-time-to-detect (MTTD) and mean-time-to-respond (MTTR).

---

#### 2. Project Scope & Objectives

##### 2.1 Core Functional Requirements

###### A. Alert Ingestion & Normalization Engine

- Ingest security alerts from heterogeneous sources: Firewall logs, EDR endpoints, Cloud SIEM (Splunk-like), Threat Intelligence feeds.
- Normalize raw alerts into a canonical Alert model regardless of source format.
- Support both push (webhooks) and pull (polling APIs) ingestion modes.

###### B. Alert Enrichment & Correlation Pipeline

- Enrich alerts with context: Geo-IP location, Threat Intel reputation scores, Asset

criticality, User identity & privilege level.

- Composite Alert Grouping: Individual alerts can be grouped into Campaigns (multi-stage attacks) or Incident Clusters — treated uniformly for analysis and response.
- Chain of enrichment: Each alert passes through a configurable pipeline of enrichment handlers.

### **C. Incident Lifecycle Management**

- Incidents transition through states: New → Under Triage → Containment → Eradication → Recovery → Post-Incident Review → Closed.
- State-dependent behavior: only certain response actions are available in specific states.
- Automated state transitions triggered by response action outcomes.

### **D. Automated Response Orchestration**

- Execute response actions: Isolate Endpoint, Block IP/Domain, Disable User Account, Quarantine File, Escalate to Tier-3 Analyst.
- Strategy-based response selection: The system selects response strategies based on incident severity, asset criticality, and business context.
- Response actions are decorated with logging, approval gates, and rollback capabilities.

### **E. Threat Intelligence Integration**

- Integrate external threat intel providers (VirusTotal, MISP, custom feeds).
- Proxy layer handles caching, rate limiting, and access control for external APIs.
- Adapter layer normalizes disparate threat intel formats.

### **F. SOC Analyst Dashboard (MVC)**

- Real-time alert stream, incident queue, response action history.
- Observer-driven updates: Dashboard receives push updates when incidents change state or new alerts arrive.
- Analyst can manually trigger response actions, override automated decisions, and annotate incidents.

### **G. Audit & Compliance Logging**

- Every enrichment, classification, and response action is immutably logged.
- Compliance reporting: GDPR breach notification timelines, SOC2 evidence collection.

## **2.2 Out-of-Scope**

- Machine-learning-based threat detection (use rule-based classification instead).
- Mobile SOC analyst app (responsive web only).
- Multi-tenant cloud SaaS deployment (single-tenant enterprise deployment).
- Payment or billing modules.

---

## **3. Team Structure & Responsibilities**

| Role                            | Student   | Primary Domain                                                                  | Patterns & Architecture Focus                                           |
|---------------------------------|-----------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Threat Pipeline Engineer        | Student A | Alert Ingestion, Enrichment Engine, Correlation, Persistence                    | Composite, State, Factory Method, Abstract Factory, Singleton, Strategy |
| Integration & Response Engineer | Student B | Threat Intel Integration, Response Orchestration, Security Middleware, Adapters | Adapter, Decorator, Proxy, Facade, Chain of Responsibility              |
| SOC Platform Engineer           | Student C | Analyst Dashboard, Event Bus, SOA Service Orchestration, Real-Time Systems      | MVC, Observer, SOA, Event-Driven, Layered Architecture                  |

---

## 4. Mandatory Pattern & Architecture Mapping

### 4.1 Design Patterns (12 Required)

| Pattern          | Location / Subsystem                                                                                                             | Natural Justification                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Singleton        | `AlertIngestionConfig`,<br>`ThreatIntelCache`, `EventBusPublisher`                                                               | Single source of truth for configuration, shared cache, and event coordination                                                       |
| Factory Method   | `AlertNormalizerFactory`,<br>`ResponseActionFactory`                                                                             | Different sources produce different alert formats; different incident types need different response actions                          |
| Abstract Factory | `EnrichmentProviderFactory`,<br>`NotifierFactory`                                                                                | Create families of related enrichment providers (GeolP + ThreatIntel + AssetDB) or notification channels (Email + Slack + PagerDuty) |
| Composite        | `Alert` → `AlertCampaign` /<br>`IncidentCluster` tree                                                                            | A Campaign contains Alerts; both must be analyzed, enriched, and responded to uniformly                                              |
| Facade           | `IncidentResponseFacade`                                                                                                         | Simplifies the complex subsystem of state validation, action execution, logging, and notification for client code                    |
| Adapter          | `SplunkAdapter`, `CrowdStrikeAdapter`,<br>`VirusTotalAdapter`, `MISPAdapter`                                                     | Convert external APIs and data formats to SDA-Pro canonical interfaces                                                               |
| Decorator        | `EnrichmentDecorator`,<br>`ApprovalGateDecorator`,<br>`AuditLogDecorator`, `RollbackDecorator`                                   | Add enrichment, approval, logging, and rollback behavior dynamically to response actions                                             |
| Proxy            | `ThreatIntelProxy` (caching + rate limiting),<br>`ResponseActionProxy` (authorization),<br>`LazyLoadingProxy` (incident details) | Control access to expensive external APIs, enforce authorization, defer heavy DB loads                                               |
| State            | `Incident` lifecycle: NewState → TriageState → ContainmentState → EradicationState → RecoveryState → ClosedState                 | Encapsulate state-specific behavior and legal transitions                                                                            |

|                         |                                                                                                                          |                                                                                     |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Chain of Responsibility | Alert processing pipeline:<br>DeduplicationHandler →<br>EnrichmentHandler →<br>ClassificationHandler → RoutingHandler    | Decouple alert processing stages; allow dynamic pipeline configuration              |
| Observer                | `IncidentEventPublisher` /<br>`DashboardUpdater`, `AuditLogger`,<br>`NotificationDispatcher`,<br>`MetricsCollector`      | Maintain consistency across loosely coupled subsystems without tight coupling       |
| Strategy                | `ResponseStrategy`:<br>AggressiveContainmentStrategy,<br>BalancedStrategy, ConservativeStrategy,<br>WatchAndWaitStrategy | Interchangeable response algorithms based on incident severity and business context |

#### 4.2 Architecture Styles (4 Required)

| Style                     | Application                                                                                                                                                                                           | Evidence Required                                                               |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| SOA                       | Core capabilities as autonomous services: Alert Ingestion Service, Enrichment & Correlation Service, Incident Management Service, Response Orchestration Service, Notification Service, Audit Service | Component diagram, service contracts, API specifications                        |
| MVC                       | SOC Analyst Dashboard                                                                                                                                                                                 | Directory structure ('Controllers/', 'Models/', 'Views/'), request flow diagram |
| Layered Architecture      | Internal structure of every SOA service                                                                                                                                                               | Clear separation: Presentation → Business Logic → Data Access                   |
| Event-Driven Architecture | Alert ingestion, incident state changes, and response completions publish domain events to an event bus                                                                                               | Event schema definitions, message broker topology, pub/sub evidence             |

---

## 5. Technical Stack Recommendations

| Layer                    | Recommended Technology                                             |
|--------------------------|--------------------------------------------------------------------|
| Backend Services         | Java (Spring Boot) or C# (.NET Core) or Node.js (NestJS)           |
| Frontend (MVC Dashboard) | React / Vue.js                                                     |
| Message Broker           | RabbitMQ or Apache Kafka                                           |
| Database                 | PostgreSQL (relational) + Redis (caching, sessions, rate limiting) |
| Real-Time                | WebSockets or Server-Sent Events (SSE)                             |

|                  |                         |
|------------------|-------------------------|
| Containerization | Docker + Docker Compose |
| Version Control  | Git with GitFlow        |

---

## 6. Milestones & Deliverables

| Milestone                      | Week | Deliverables                                                                                                                                                      | Weight |
|--------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| M1: Threat Domain Design       | 3    | UML Class/Component/Sequence diagrams; ADRs; API contracts; Canonical alert schema; Composite + State pattern prototypes                                          | 15%    |
| M2: Pipeline & Engine          | 6    | Working alert ingestion (2 adapters); Enrichment pipeline (Chain of Responsibility + Decorator); Incident state machine; Factory + Strategy for response actions  | 20%    |
| M3: Integration & Intelligence | 10   | Threat intel integration (Proxy + Adapter); Response orchestration (Facade + Decorator + Proxy); Event bus (Observer) with 3+ subscribers; Dashboard MVC scaffold | 25%    |
| M4: System Integration         | 13   | End-to-end demo: Ingest alert → Enrich → Classify → Create Incident → Auto-respond → Dashboard updates → Audit log; Docker Compose deployment; Integration tests  | 25%    |
| M5: Final Delivery             | 16   | Final codebase, 20-min presentation, updated UML, ADRs, individual reflection reports on pattern usage                                                            | 15%    |

---

## 7. Architecture Decision Records (ADRs) — Required

1. **ADR-001:** SOA vs. Microservices vs. Modular Monolith — justify service boundaries
2. **ADR-002:** Synchronous vs. Asynchronous inter-service communication — hybrid approach
3. **ADR-003:** Database strategy: Why relational (PostgreSQL) for incidents + document store for alert raw data
4. **ADR-004:** Threat intel caching strategy: In-memory vs. Redis vs. external cache
5. **ADR-005:** Real-time push strategy: WebSockets vs. SSE vs. polling

---

## 8. Risk Assessment



| Risk                                               | Probability | Impact | Mitigation                                                  |
|----------------------------------------------------|-------------|--------|-------------------------------------------------------------|
| Integration failure between ingestion and response | Medium      | High   | Define canonical alert schema in Week 1; use contract tests |
| Over-engineering with patterns                     | High        | Medium | Graded on justification — forced patterns penalized         |
| External API rate limiting (VirusTotal, etc.)      | High        | Medium | Proxy layer with caching; mock adapters for testing         |
| Uneven workload                                    | Medium      | Medium | Weekly stand-ups; peer evaluation component                 |

---

## 9. Acceptance Criteria

- [ ] Alerts from 2+ sources (e.g., Webhook + API poll) are ingested and normalized.
- [ ] Alerts are enriched through a Chain of Responsibility pipeline (min. 3 handlers).
- [ ] Composite grouping: Alerts can be grouped into Campaigns and treated uniformly.
- [ ] Incidents transition through all 6 states with state-specific behavior.
- [ ] At least 2 response strategies are selectable and executable.
- [ ] At least 2 external threat intel adapters are integrated via Proxy + Adapter.
- [ ] Dashboard updates in real-time (< 2 seconds) via Observer/Event-Driven pattern.
- [ ] All 12 design patterns are identified in code with `// PATTERN: Name` annotations.
- [ ] All 4 architecture styles are documented with diagrams and directory evidence.
- [ ] System starts with `docker-compose up` and passes end-to-end integration test.

---

## 10. Grading Rubric

| Criteria                  | Weight | Excellent (90–100)                                                                                  | Satisfactory (70–89)                                               | Needs Improvement (<70)                                          |
|---------------------------|--------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------|
| Architectural Correctness | 25%    | Clean SOA boundaries; MVC strictly followed; clear layered separation; event bus properly decouples | Minor boundary violations; MVC mostly correct; some layering leaks | Monolithic tendencies; MVC confusion; tight coupling             |
| Pattern Sophistication    | 25%    | Patterns emerge naturally from requirements; strong justifications; elegant implementations         | All patterns present; some slightly forced but justified           | Patterns shoehorned; incorrect implementations; missing patterns |
| Integration Quality       | 20%    | Seamless end-to-end flow; robust error handling across services; clean contracts                    | Works end-to-end with minor edge-case issues                       | Frequent integration failures; unclear contracts                 |
| Code Quality &            | 15%    | New alert source or response action added without core                                              | Mostly extensible; reasonable test                                 | Brittle code; core modifications                                 |

|                              |     |                                                                      |                                                       |                                                   |
|------------------------------|-----|----------------------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------|
| Extensibility                |     | changes; comprehensive tests                                         | coverage                                              | needed for extensions                             |
| Presentation & Documentation | 15% | Clear UML; insightful ADRs; confident Q&A; live demo without scripts | Adequate diagrams; basic ADRs; demo with minor issues | Missing diagrams; poor justifications; demo fails |

-----

# Part II: Repository Guide

## SDA-Pro — Repository Structure & Developer Guide

### 1. Repository Organization (Poly-repo Simulation)



```

├── tests/
├── Dockerfile

├── response-orchestration-service/ # Student B (primary)
│   ├── src/
│   │   ├── controllers/
│   │   ├── services/
│   │   │   ├── facade/ # Facade: IncidentResponseFacade
│   │   │   ├── strategy/ # Strategy: response algorithms
│   │   │   ├── decorator/ # Decorator: audit, approval, rollback
│   │   │   └── executor/ # Response action execution
│   │   ├── domain/
│   │   │   ├── action/ # ResponseAction entities
│   │   │   └── outcome/ # ActionOutcome, RollbackContext
│   │   └── config/
│   ├── tests/
│   └── Dockerfile

├── threat-intel-service/ # Student B (primary)
│   ├── src/
│   │   ├── controllers/
│   │   ├── services/
│   │   │   ├── adapter/ # Adapter: VirusTotal, MISP, custom feeds
│   │   │   ├── proxy/ # Proxy: caching, rate limiting, auth
│   │   │   └── cache/ # Threat intel cache management
│   │   ├── domain/
│   │   │   ├── intel/ # ThreatIndicator entities
│   │   │   └── reputation/ # ReputationScore, Verdict
│   │   └── config/
│   ├── tests/
│   └── Dockerfile

├── notification-service/ # Student B & C
│   ├── src/
│   │   ├── channels/ # Abstract Factory families: Email, Slack, Page
│   │   ├── services/
│   │   └── controllers/
│   ├── tests/
│   └── Dockerfile

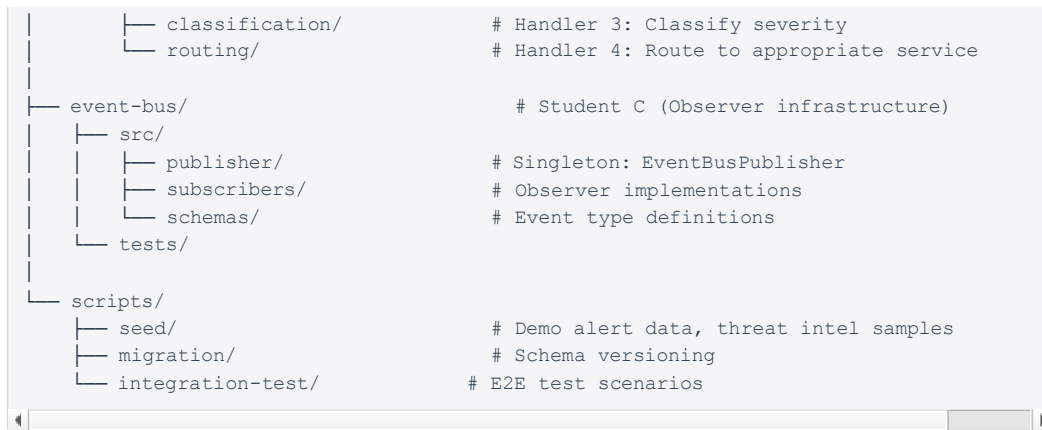
├── audit-service/ # Student C
│   ├── src/
│   │   ├── controllers/
│   │   ├── services/ # Immutable audit logging, compliance reports
│   │   └── repositories/
│   ├── tests/
│   └── Dockerfile

├── identity-service/ # Student C
│   ├── src/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── tests/
│   └── Dockerfile

├── soc-dashboard/ # Student C (MVC Application)
│   ├── src/
│   │   ├── controllers/ # MVC: Route handlers
│   │   ├── models/ # MVC: API clients, DTOs, state
│   │   ├── views/ # MVC: React/Vue components
│   │   ├── services/ # Dashboard-specific business logic
│   │   └── store/ # State management
│   ├── tests/
│   └── Dockerfile

├── middleware/ # Student B (Chain of Responsibility)
│   ├── src/
│   │   ├── deduplication/ # Handler 1: Deduplicate repeated alerts
│   │   └── enrichment/ # Handler 2: Enrich with context

```



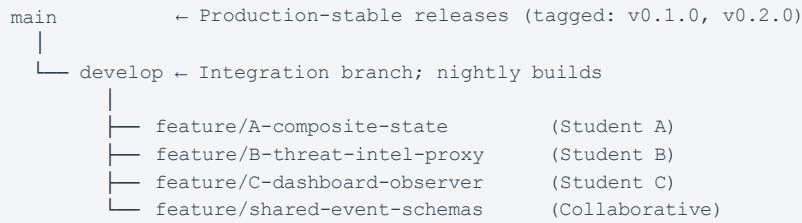
---

2. Module-to-Student Mapping

| Directory                                 | Primary Owner   | Supporting Owner | Patterns Owned                                                 |
|-------------------------------------------|-----------------|------------------|----------------------------------------------------------------|
| `services/alert-ingestion-service`        | Student A       | —                | Singleton, Composite, Factory Method                           |
| `services/enrichment-correlation-service` | Student A       | —                | Chain of Responsibility, Composite, Strategy, Abstract Factory |
| `services/incident-management-service`    | Student A       | —                | State                                                          |
| `services/response-orchestration-service` | Student B       | —                | Facade, Strategy, Decorator                                    |
| `services/threat-intel-service`           | Student B       | —                | Adapter, Proxy                                                 |
| `services/notification-service`           | Student B       | Student C        | Abstract Factory                                               |
| `services/audit-service`                  | Student C       | —                | Observer (consumer)                                            |
| `services/identity-service`               | Student C       | Student B        | —                                                              |
| `soc-dashboard/`                          | Student C       | —                | MVC                                                            |
| `middleware/`                             | Student B       | Student A        | Chain of Responsibility                                        |
| `event-bus/`                              | Student C       | Student B        | Observer, Singleton                                            |
| `shared/`                                 | All (consensus) | —                | —                                                              |

---

3. Git Branching Strategy (GitFlow)



#### 4. API Contracts Between Services

##### Synchronous (REST/gRPC)

- Dashboard → Identity: `GET /identity/analysts/{id}`
- Dashboard → Incident Management: `GET /incidents`, `POST /incidents/{id}/actions`
- Response Orchestration → Threat Intel: `POST /threat-intel/reputation` (check IP/file hash)
- Response Orchestration → Notification: `POST /notifications/dispatch`

##### Asynchronous (Event-Driven / Observer)

| Event Type               | Publisher              | Subscribers                                     |
|--------------------------|------------------------|-------------------------------------------------|
| `AlertIngested`          | Alert Ingestion        | Enrichment, Dashboard, Audit                    |
| `AlertEnriched`          | Enrichment             | Correlation, Dashboard, Audit                   |
| `IncidentCreated`        | Incident Management    | Dashboard, Notification, Response Orchestration |
| `IncidentStateChanged`   | Incident Management    | Dashboard, Audit, Metrics                       |
| `ResponseActionExecuted` | Response Orchestration | Dashboard, Audit, Notification                  |
| `ThreatIntelUpdated`     | Threat Intel           | Enrichment (cache invalidation)                 |

#### 5. Testing Strategy

| Test Type         | Scope                                      | Responsibility                 |
|-------------------|--------------------------------------------|--------------------------------|
| Unit Tests        | Individual classes (patterns in isolation) | Each student for their modules |
| Integration Tests | Service + DB + Message Bus                 | Rotating pair each week        |
| Contract Tests    | API request/response shapes                | Provider service owner         |
| End-to-End Tests  | Full `docker-compose` stack                | All 3 together (M4)            |

---

## 6. Pattern Annotation Convention

```
// PATTERN: State
// RATIONALE: Incident behavior and legal transitions vary by lifecycle phase.
public class ContainmentState implements IncidentState { ... }
```

---





## Part III–VI: UML Diagrams

To render these diagrams, paste the code into [plantuml.com/plantuml](https://plantuml.com/plantuml) or use a PlantUML plugin.

### UML Class Diagram — All 12 Patterns

```
@startuml SDA-Pro_Class_Diagram
!theme plain
skinparam packageStyle rectangle
skinparam linetype ortho

title SDA-Pro – Comprehensive Class Diagram (All 12 Patterns) '

=====
' PACKAGE: domain.alert – Composite Pattern
' =====
package "domain.alert (Composite)" #LightCoral {

    interface AlertComponent {
        +getId(): UUID
        +getSeverity(): Severity
        +getTimestamp(): Instant
        +accept(visitor: AlertVisitor): void
        +add(component: AlertComponent): void
        +remove(component: AlertComponent): void
        +getChildren(): List<AlertComponent>
    }

    abstract class AbstractAlert {
        -id: UUID
        -severity: Severity
        -timestamp: Instant
        -source: AlertSource
        +getId(): UUID
        +getSeverity(): Severity
    }

    class SingleAlert {
        -rawPayload: JsonNode
        -normalizedData: CanonicalAlert
        +accept(visitor: AlertVisitor): void
    }

    class AlertCampaign {
        -campaignName: String
        -attackPattern: String
        -children: List<AlertComponent>
        +add(component: AlertComponent): void
        +remove(component: AlertComponent): void
        +getChildren(): List<AlertComponent>
        +accept(visitor: AlertVisitor): void
        +getMaxSeverity(): Severity
    }

    class IncidentCluster {
        -clusterId: UUID
        -correlationRule: String
        -children: List<AlertComponent>
        +add(component: AlertComponent): void
        +remove(component: AlertComponent): void
        +getChildren(): List<AlertComponent>
        +accept(visitor: AlertVisitor): void
    }

    AlertComponent <|-- AbstractAlert
    AbstractAlert <|-- SingleAlert
}
```

```

AbstractAlert <|-- AlertCampaign
AbstractAlert <|-- IncidentCluster
AlertCampaign "0..*" o-- "1" AlertComponent : children >
IncidentCluster "0..*" o-- "1" AlertComponent : children >
}

' =====
' PACKAGE: domain.incident - State Pattern
' =====
package "domain.incident (State)" #LightBlue {

    class Incident {
        -id: UUID
        -currentState: IncidentState
        -affectedAlerts: AlertComponent
        -createdAt: Instant
        +setState(state: IncidentState): void
        +beginTriage(analystId: UUID): void
        +initiateContainment(actions: List<ResponseAction>): void
        +beginEradication(): void
        +beginRecovery(): void
        +close(report: PostIncidentReport): void
        +escalate(reason: String): void
        +getState(): IncidentState
    }

    interface IncidentState {
        +beginTriage(incident: Incident, analystId: UUID): void
        +initiateContainment(incident: Incident, actions: List<ResponseAction>): void
        +beginEradication(incident: Incident): void
        +beginRecovery(incident: Incident): void
        +close(incident: Incident, report: PostIncidentReport): void
        +escalate(incident: Incident, reason: String): void
        +getName(): String
        +getAllowedActions(): List<ResponseActionType>
    }

    class NewState
    class UnderTriageState
    class ContainmentState
    class EradicationState
    class RecoveryState
    class PostIncidentReviewState
    class ClosedState

    Incident --> IncidentState
    IncidentState <|.. NewState
    IncidentState <|.. UnderTriageState
    IncidentState <|.. ContainmentState
    IncidentState <|.. EradicationState
    IncidentState <|.. RecoveryState
    IncidentState <|.. PostIncidentReviewState
    IncidentState <|.. ClosedState
}

' =====
' PACKAGE: execution.response - Strategy + Factory + Decorator + Facade
' =====
package "execution.response (Strategy + Factory + Decorator + Facade)" #LightGreen {

    interface ResponseStrategy {
        +determineActions(incident: Incident, context: ResponseContext): List<ResponseActi
        +getName(): String
        +getDescription(): String
    }

    class AggressiveContainmentStrategy
    class BalancedResponseStrategy
    class ConservativeStrategy
    class WatchAndWaitStrategy

```

```

class ResponseActionFactory {
  +{static} createAction(type: ResponseActionType, params: Map): ResponseAction
}

interface ResponseAction {
  +execute(target: TargetAsset): ActionOutcome
  +getType(): ResponseActionType
  +isReversible(): boolean
  +rollback(): RollbackOutcome
}

class IsolateEndpointAction
class BlockIPAction
class DisableUserAction
class QuarantineFileAction
class EscalateAction

abstract class ResponseActionDecorator {
  -wrappedAction: ResponseAction
  +execute(target: TargetAsset): ActionOutcome
  +rollback(): RollbackOutcome
}

class ApprovalGateDecorator {
  -approvalService: ApprovalService
  +execute(target: TargetAsset): ActionOutcome
}

class AuditLogDecorator {
  -auditService: AuditService
  +execute(target: TargetAsset): ActionOutcome
}

class RollbackCapabilityDecorator {
  -snapshotService: SnapshotService
  +execute(target: TargetAsset): ActionOutcome
  +rollback(): RollbackOutcome
}

class MetricsDecorator {
  -metricsService: MetricsService
  +execute(target: TargetAsset): ActionOutcome
}

class IncidentResponseFacade {
  -incidentService: IncidentService
  -strategySelector: ResponseStrategySelector
  -actionFactory: ResponseActionFactory
  -eventPublisher: EventBusPublisher
  +assessAndRespond(incidentId: UUID): ResponsePlan
  +executeResponsePlan(plan: ResponsePlan): ExecutionResult
  +rollbackActions(incidentId: UUID): RollbackResult
  +getResponseHistory(incidentId: UUID): List<ActionOutcome>
}

ResponseStrategy <|.. AggressiveContainmentStrategy
ResponseStrategy <|.. BalancedResponseStrategy
ResponseStrategy <|.. ConservativeStrategy
ResponseStrategy <|.. WatchAndWaitStrategy

ResponseAction <|.. IsolateEndpointAction
ResponseAction <|.. BlockIPAction
ResponseAction <|.. DisableUserAction
ResponseAction <|.. QuarantineFileAction
ResponseAction <|.. EscalateAction

ResponseActionFactory ..> ResponseAction : creates >

ResponseAction <|-- ResponseActionDecorator

```

```

ResponseActionDecorator "1" *-- "1" ResponseAction : wraps >
ResponseActionDecorator <|-- ApprovalGateDecorator
ResponseActionDecorator <|-- AuditLogDecorator
ResponseActionDecorator <|-- RollbackCapabilityDecorator
ResponseActionDecorator <|-- MetricsDecorator

IncidentResponseFacade --> ResponseStrategy
IncidentResponseFacade --> ResponseActionFactory
IncidentResponseFacade --> Incident
}

' =====
' PACKAGE: ingestion - Factory Method + Singleton
' =====
package "ingestion (Factory Method + Singleton)" #LightYellow {

    interface AlertNormalizer {
        +normalize(rawPayload: JsonNode): CanonicalAlert
        +supports(sourceType: AlertSourceType): boolean
    }

    class SplunkNormalizer
    class CrowdStrikeNormalizer
    class FirewallNormalizer
    class CloudSIEMNormalizer

    class AlertNormalizerFactory {
        -(static) normalizers: Map<AlertSourceType, AlertNormalizer>
        +(static) register(type: AlertSourceType, normalizer: AlertNormalizer): void
        +(static) createNormalizer(sourceType: AlertSourceType): AlertNormalizer
    }

    class IngestionConfigManager {
        -(static) instance: IngestionConfigManager
        -sources: List<AlertSourceConfig>
        -IngestionConfigManager()
        +(static) getInstance(): IngestionConfigManager
        +getSourceConfig(type: AlertSourceType): AlertSourceConfig
        +isIngestionEnabled(type: AlertSourceType): boolean
    }

    AlertNormalizer <|.. SplunkNormalizer
    AlertNormalizer <|.. CrowdStrikeNormalizer
    AlertNormalizer <|.. FirewallNormalizer
    AlertNormalizer <|.. CloudSIEMNormalizer
    AlertNormalizerFactory ..> AlertNormalizer : creates >
}

' =====
' PACKAGE: enrichment - Chain of Responsibility + Abstract Factory
' =====
package "enrichment (Chain of Responsibility + Abstract Factory)" #MistyRose {

    abstract class EnrichmentHandler {
        -nextHandler: EnrichmentHandler
        +setNext(handler: EnrichmentHandler): EnrichmentHandler
        +handle(alert: AlertComponent): EnrichmentResult
        +(abstract) doEnrich(alert: AlertComponent): EnrichmentResult
    }

    class DeduplicationHandler {
        -alertCache: AlertCache
        +doEnrich(alert: AlertComponent): EnrichmentResult
    }

    class GeoIPHandler {
        -geoService: GeoLocationService
        +doEnrich(alert: AlertComponent): EnrichmentResult
    }
}

```

```

class ThreatIntelHandler {
    -intelService: ThreatIntelService
    +doEnrich(alert: AlertComponent): EnrichmentResult
}

class AssetContextHandler {
    -assetService: AssetInventoryService
    +doEnrich(alert: AlertComponent): EnrichmentResult
}

class ClassificationHandler {
    -classifier: RuleBasedClassifier
    +doEnrich(alert: AlertComponent): EnrichmentResult
}

EnrichmentHandler <|-- DeduplicationHandler
EnrichmentHandler <|-- GeoIPHandler
EnrichmentHandler <|-- ThreatIntelHandler
EnrichmentHandler <|-- AssetContextHandler
EnrichmentHandler <|-- ClassificationHandler
EnrichmentHandler "0..1" --> "0..1" EnrichmentHandler : next >

interface EnrichmentProviderFactory {
    +createGeoProvider(): GeoLocationService
    +createThreatIntelProvider(): ThreatIntelService
    +createAssetProvider(): AssetInventoryService
}

class PremiumEnrichmentFactory
class StandardEnrichmentFactory

EnrichmentProviderFactory <|.. PremiumEnrichmentFactory
EnrichmentProviderFactory <|.. StandardEnrichmentFactory
}

' =====
' PACKAGE: threat-intel - Adapter + Proxy
' =====
package "threat-intel (Adapter + Proxy)" #LightBlue {

    interface ThreatIntelProvider {
        +checkReputation(indicator: String, type: IndicatorType): ReputationResult
        +getIndicators(feedType: FeedType): List<ThreatIndicator>
    }

    class VirusTotalAdapter {
        -vtClient: VirusTotalAPI
        +checkReputation(indicator: String, type: IndicatorType): ReputationResult
    }

    class MISPAdapter {
        -mispClient: MISPAPI
        +checkReputation(indicator: String, type: IndicatorType): ReputationResult
    }

    class CustomFeedAdapter {
        -httpClient: HttpClient
        +checkReputation(indicator: String, type: IndicatorType): ReputationResult
    }

    interface ThreatIntelProxy {
        +checkReputation(indicator: String, type: IndicatorType): ReputationResult
    }

    class CachingProxy {
        -realProvider: ThreatIntelProvider
        -cache: ThreatIntelCache
        +checkReputation(indicator: String, type: IndicatorType): ReputationResult
    }
}

```

```

class RateLimitProxy {
    -realProvider: ThreatIntelProvider
    -rateLimiter: RateLimiter
    +checkReputation(indicator: String, type: IndicatorType): ReputationResult
}

class AccessControlProxy {
    -realProvider: ThreatIntelProvider
    -securityManager: SecurityManager
    +checkReputation(indicator: String, type: IndicatorType): ReputationResult
}

ThreatIntelProvider <|.. VirusTotalAdapter
ThreatIntelProvider <|.. MISPAdapter
ThreatIntelProvider <|.. CustomFeedAdapter

ThreatIntelProxy <|.. CachingProxy
ThreatIntelProxy <|.. RateLimitProxy
ThreatIntelProxy <|.. AccessControlProxy

CachingProxy "1" --> "1" ThreatIntelProvider : delegates >
RateLimitProxy "1" --> "1" ThreatIntelProvider : delegates >
AccessControlProxy "1" --> "1" ThreatIntelProvider : delegates >
}

' =====
' PACKAGE: notification - Abstract Factory
' =====
package "notification (Abstract Factory)" #Honeydew {

    interface NotificationFactory {
        +createEmailNotifier(): EmailNotifier
        +createSlackNotifier(): SlackNotifier
        +createPagerDutyNotifier(): PagerDutyNotifier
    }

    class EnterpriseNotificationFactory
    class BasicNotificationFactory

    interface Notifier {
        +send(notification: Notification): DeliveryResult
    }

    class EmailNotifier
    class SlackNotifier
    class PagerDutyNotifier

    NotificationFactory <|.. EnterpriseNotificationFactory
    NotificationFactory <|.. BasicNotificationFactory

    Notifier <|.. EmailNotifier
    Notifier <|.. SlackNotifier
    Notifier <|.. PagerDutyNotifier

    EnterpriseNotificationFactory ..> EmailNotifier : creates >
    EnterpriseNotificationFactory ..> SlackNotifier : creates >
    EnterpriseNotificationFactory ..> PagerDutyNotifier : creates >
}

' =====
' PACKAGE: events - Observer + Singleton
' =====
package "events (Observer + Singleton)" #LightGray {

    interface Subject {
        +attach(observer: Observer): void
        +detach(observer: Observer): void
        +notify(event: DomainEvent): void
    }
}

```

```

interface Observer {
    +update(event: DomainEvent): void
    +getObserverId(): String
}

class EventBusPublisher {
    -(static) instance: EventBusPublisher
    -observers: Map<String, List<Observer>>
    -EventBusPublisher()
    +(static) getInstance(): EventBusPublisher
    +attach(eventType: String, observer: Observer): void
    +detach(eventType: String, observer: Observer): void
    +notify(event: DomainEvent): void
    +publishAlertIngested(alert: AlertComponent): void
    +publishIncidentStateChanged(incident: Incident): void
    +publishResponseActionExecuted(action: ResponseAction): void
}

class DashboardUpdater {
    -websocketManager: WebSocketManager
    +update(event: DomainEvent): void
}

class AuditEventLogger {
    -auditRepository: AuditRepository
    +update(event: DomainEvent): void
}

class NotificationDispatcher {
    -notificationService: NotificationService
    +update(event: DomainEvent): void
}

class MetricsCollector {
    -metricsRepository: MetricsRepository
    +update(event: DomainEvent): void
}

Subject <|.. EventBusPublisher
Observer <|.. DashboardUpdater
Observer <|.. AuditEventLogger
Observer <|.. NotificationDispatcher
Observer <|.. MetricsCollector
EventBusPublisher "1" o-- "0..*" Observer : observers >
}

' =====
' PACKAGE: presentation - MVC
' =====
package "presentation (MVC)" #AliceBlue {

    class AlertStreamController {
        -alertService: AlertService
        +getAlertStream(filters: AlertFilter): SseEmitter
        +acknowledgeAlert(alertId: UUID): Response
    }

    class IncidentController {
        -incidentService: IncidentService
        +getIncidents(queue: QueueType): Response
        +updateIncidentState(id: UUID, action: StateAction): Response
        +executeManualAction(id: UUID, action: ResponseAction): Response
    }

    class DashboardController {
        -dashboardService: DashboardService
        +getMetrics(): Response
        +getRealtimeUpdates(): SseEmitter
    }
}

```

```

class AlertStreamModel {
    -alerts: List<AlertComponent>
    +toViewData(): AlertStreamViewData
}

class IncidentQueueModel {
    -incidents: List<Incident>
    +toViewData(): IncidentQueueViewData
}

class DashboardMetricsModel {
    -metrics: SOCMetrics
    +toViewData(): DashboardViewData
}

class AlertStreamView
class IncidentQueueView
class ResponseConsoleView
class MetricsDashboardView

AlertStreamController --> AlertStreamModel
IncidentController --> IncidentQueueModel
DashboardController --> DashboardMetricsModel
AlertStreamModel ..> AlertStreamView
IncidentQueueModel ..> IncidentQueueView
DashboardMetricsModel ..> MetricsDashboardView
}

' =====
' CROSS-PACKAGE RELATIONSHIPS
' =====

Incident --> AlertComponent : aggregates >
IncidentResponseFacade --> Incident : orchestrates >
IncidentResponseFacade --> EventBusPublisher : publishes >
EnrichmentHandler --> AlertComponent : enriches >
ThreatIntelHandler --> ThreatIntelProxy : queries >
AlertNormalizerFactory ..> AlertComponent : produces >
EventBusPublisher ..> DashboardUpdater : notifies >
EventBusPublisher ..> AuditEventLogger : notifies >
EventBusPublisher ..> NotificationDispatcher : notifies >

@enduml

```



## UML Component & Deployment Diagram

```
@startuml SDA-Pro_Component_Diagram
!theme plain
skinparam componentStyle uml2

title SDA-Pro - SOA Component & Deployment Architecture

' =====
' EXTERNAL ACTORS & SYSTEMS
' =====
actor "SOC Analyst" as Analyst
actor "Security Engineer" as Engineer

cloud "External Security Infrastructure" {
    [Splunk SIEM] as Splunk
    [CrowdStrike EDR] as CrowdStrike
    [Palo Alto Firewall] as Firewall
    [VirusTotal API] as VT
    [MISP Platform] as MISP
}

cloud "Enterprise Systems" {
    [Active Directory] as AD
    [Email Gateway] as Email
    [Slack Workspace] as Slack
    [PagerDuty] as PagerDuty
}

' =====
' INFRASTRUCTURE LAYER
' =====
node "Message Bus" #LightGray {
    [RabbitMQ / Kafka] as MessageBus
}

node "Data Layer" #LightGray {
    database "PostgreSQL" as Postgres {
        [Incident Records] as IncDB
        [Alert Archive] as AlertDB
        [Audit Logs] as AuditDB
    }
    database "Redis" as Redis {
        [Threat Intel Cache] as TICache
        [Session Store] as SessionCache
        [Rate Limit Counters] as RateCache
    }
}

' =====
' SOA SERVICES
' =====
package "Service-Oriented Architecture (SOA) Layer" #LightBlue {
    [Alert Ingestion Service] as AIS <<Service>> #LightCoral
    [Enrichment & Correlation Service] as ECS <<Service>> #LightGreen
    [Incident Management Service] as IMS <<Service>> #LightYellow
    [Response Orchestration Service] as ROS <<Service>> #LightBlue
    [Threat Intel Service] as TIS <<Service>> #LightPink
    [Notification Service] as NS <<Service>> #Honeydew
    [Audit Service] as AS <<Service>> #MistyRose
    [Identity Service] as IS <<Service>> #LightGray
}

' =====
' PRESENTATION LAYER (MVC)
' =====
```

```

package "SOC Dashboard (MVC)" #AliceBlue {

    [View Layer
    React/Vue Components] as ViewLayer
    [Controller Layer
    REST/WS Handlers] as ControllerLayer
    [Model Layer
    API Clients + DTOs] as ModelLayer
}

' =====
' MIDDLEWARE & CROSS-CUTTING
' =====

package "Alert Processing Pipeline (Chain of Responsibility)" #MistyRose {
    [Deduplication Handler] as MH1
    [Geo-IP Handler] as MH2
    [Threat Intel Handler] as MH3
    [Asset Context Handler] as MH4
    [Classification Handler] as MH5
}

package "Event Subscribers (Observer)" #Honeydew {
    [Dashboard Updater] as Obs1
    [Audit Logger] as Obs2
    [Notification Dispatcher] as Obs3
    [Metrics Collector] as Obs4
}

' =====
' RELATIONSHIPS - INGESTION (External → Internal)
' =====

Splunk ..> AIS : webhook / API poll
CrowdStrike ..> AIS : streaming API
Firewall ..> AIS : syslog / webhook

' =====
' RELATIONSHIPS - SYNCHRONOUS
' =====

Analyst ..> ViewLayer : monitors & responds
Engineer ..> ViewLayer : configures rules

ViewLayer --> ControllerLayer : HTTP / WebSocket
ControllerLayer --> ModelLayer : data binding

ControllerLayer --> AIS : REST (alert queries)
ControllerLayer --> IMS : REST (incident CRUD)
ControllerLayer --> ROS : REST (manual actions)
ControllerLayer --> AS : REST (audit reports)
ControllerLayer --> IS : REST (auth)

AIS --> ECS : gRPC (send normalized alerts)
ECS --> IMS : gRPC (create incidents)
ROS --> TIS : gRPC (reputation checks)
ROS --> NS : REST (dispatch notifications)

' =====
' RELATIONSHIPS - MIDDLEWARE PIPELINE
' =====

AIS --> MH1 : raw alerts
MH1 --> MH2
MH2 --> MH3
MH3 --> MH4
MH4 --> MH5
MH5 --> ECS : enriched alerts

' =====
' RELATIONSHIPS - THREAT INTEL EXTERNAL
' =====

TIS --> VT : HTTPS (Adapter)
TIS --> MISP : HTTPS (Adapter)

```

```

TIS --> Redis : cache lookups

' =====
' RELATIONSHIPS - NOTIFICATION EXTERNAL
' =====
NS --> Email : SMTP (Adapter)
NS --> Slack : HTTPS (Adapter)
NS --> PagerDuty : HTTPS (Adapter)

' =====
' RELATIONSHIPS - ASYNCHRONOUS / EVENT-DRIVEN
' =====
AIS ..> MessageBus : AlertIngested
ECS ..> MessageBus : AlertEnriched, IncidentCorrelated
IMS ..> MessageBus : IncidentCreated, IncidentStateChanged
ROS ..> MessageBus : ResponseActionExecuted
TIS ..> MessageBus : ThreatIntelUpdated

MessageBus ..> Obs1 : push to dashboard
MessageBus ..> Obs2 : write audit trail
MessageBus ..> Obs3 : trigger alerts
MessageBus ..> Obs4 : update metrics

Obs1 --> ViewLayer : WebSocket/SSE push
Obs2 --> AS : persist audit
Obs3 --> NS : dispatch notification
Obs4 --> AS : write time-series

' =====
' DATA ACCESS
' =====
AIS ..> AlertDB : writes normalized alerts
ECS ..> AlertDB : reads/writes enriched data
ECS ..> IncDB : writes incident clusters
IMS ..> IncDB : reads/writes incident lifecycle
AS ..> AuditDB : immutable writes
IS ..> Redis : session tokens
TIS ..> TICache : cache reputation scores

' =====
' ARCHITECTURAL ANNOTATIONS
' =====
note right of AIS
  <b>Alert Ingestion Service</b>
  • Singleton: IngestionConfigManager
  • Factory Method: AlertNormalizerFactory
  • Composite: Alert tree structure
end note

note right of ECS
  <b>Enrichment & Correlation</b>
  • Chain of Responsibility: 5 enrichment handlers
  • Composite: Campaign/Cluster traversal
  • Strategy: Correlation algorithms
  • Abstract Factory: EnrichmentProviderFactory
end note

note right of IMS
  <b>Incident Management</b>
  • State: 7 incident lifecycle states
    with state-specific behavior
end note

note right of ROS
  <b>Response Orchestration</b>
  • Facade: IncidentResponseFacade
  • Strategy: 4 response strategies
  • Decorator: Audit, Approval, Rollback
  • Proxy: Access control for actions
end note

```

```
note right of TIS
  <b>Threat Intel</b>
  • Adapter: VirusTotal, MISP, custom feeds
  • Proxy: Cache, RateLimit, AccessControl
end note

note bottom of MessageBus
  <b>Event-Driven Backbone</b>
  Physical implementation of Observer pattern.
  Publishers do not know subscribers.
end note

note left of MH1
  <b>Chain of Responsibility</b>
  Deduplicate → GeoIP → ThreatIntel
  → AssetContext → Classification
end note

@enduml
```

## UML Sequence — Alert Ingestion & Enrichment

```
@startuml SDA-Pro_Sequence_Ingestion
!theme plain
autonumber

title Sequence: Alert Ingestion → Enrichment → Incident Creation
(Composite, Factory Method, Chain of Responsibility, State, Observer)

actor "Splunk SIEM" as Splunk
participant "AlertIngestionService" as AIS
participant "AlertNormalizerFactory
(Factory Method)" as Fact
participant "SplunkNormalizer" as Norm
participant "AlertCampaign
(Composite)" as Campaign
participant "SingleAlert" as Alert
participant "EnrichmentPipeline
(CoR Assembler)" as Pipe
participant "DeduplicationHandler
(CoR: H1)" as H1
participant "GeoIPHandler
(CoR: H2)" as H2
participant "ThreatIntelHandler
(CoR: H3)" as H3
participant "ClassificationHandler
(CoR: H4)" as H4
participant "ThreatIntelProxy
(Proxy)" as Proxy
participant "VirusTotalAdapter
(Adapter)" as VTAd
participant "EnrichmentService" as ECS
participant "CorrelationStrategy
(Strategy)" as Strat
participant "IncidentManagementService" as IMS
participant "NewState
(State)" as NewSt
participant "Incident" as Incident
participant "EventBusPublisher
(Singleton + Observer)" as EB
participant "DashboardUpdater
(Observer)" as DashObs
participant "AuditLogger
(Observer)" as AuditObs

database "PostgreSQL" as DB
queue "RabbitMQ" as MQ

== 1. Alert Ingestion ==
Splunk -> AIS : POST /ingest/splunk (webhook)
AIS -> AIS : validateSourceAuth()

== 2. Factory Method - Normalize ==
AIS -> Fact : createNormalizer(SPLUNK)
Fact -> Norm : new SplunkNormalizer()
Fact --> AIS : normalizer

AIS -> Norm : normalize(rawPayload)
Norm -> Norm : mapSplunkSchemaToCanonical()
Norm --> AIS : CanonicalAlert

== 3. Composite - Group into Campaign ==
AIS -> Campaign : new AlertCampaign("APT29-Campaign")
AIS -> Campaign : add(alert)
Campaign -> Alert : new SingleAlert(canonicalData)
Campaign --> AIS : compositeAlert
```

```

== 4. Chain of Responsibility - Enrichment Pipeline ==
AIS -> Pipe : assemblePipeline()
Pipe -> H1 : setNext(H2)
H1 -> H2 : setNext(H3)
H2 -> H3 : setNext(H4)

AIS -> H1 : handle(campaign)

H1 -> H1 : checkDuplicate(campaign)
H1 -> H1 : not duplicate
H1 -> H2 : doHandle(campaign)

H2 -> H2 : enrichGeoIP(campaign)
H2 -> H2 : addLocationContext()
H2 -> H3 : doHandle(campaign)

H3 -> Proxy : checkReputation(ip, IP_ADDRESS)
Proxy -> Proxy : checkCache()
alt Cache miss
    Proxy -> VTAd : queryVirusTotal(ip)
    VTAd -> VTAd : mapToCanonicalFormat()
    VTAd --> Proxy : reputationScore
    Proxy -> Proxy : cacheResult()
else Cache hit
    Proxy --> H3 : cachedReputation
end
Proxy --> H3 : ReputationResult.MALICIOUS
H3 -> H3 : addThreatContext()
H3 -> H4 : doHandle(campaign)

H4 -> H4 : classifySeverity(campaign)
H4 -> H4 : severity = CRITICAL
H4 --> Pipe : EnrichmentResult.COMPLETE

== 5. Strategy - Correlation ==
Pipe --> AIS : enrichedCampaign
AIS -> ECS : processEnrichedAlert(campaign)
ECS -> Strat : correlate(enrichedCampaign)
Strat -> Strat : patternMatch("lateral-movement")
Strat -> Strat : exceedsThreshold(3, CRITICAL)
Strat --> ECS : CorrelationResult.CREATE_INCIDENT

== 6. State - Incident Creation ==
ECS -> IMS : createIncident(campaign, correlationResult)
IMS -> Incident : new Incident(campaign)
Incident -> Incident : setState(NewState)
NewSt -> NewSt : onEnter(incident)
NewSt -> NewSt : scheduleAutoTriage(incident, 5min)
Incident --> IMS : incident

IMS -> DB : INSERT incident

== 7. Observer - Event Publication ==
IMS -> EB : publishIncidentCreated(incident)
EB -> MQ : push "IncidentCreated" event

par Parallel Observer Reactions
    MQ -> DashObs : consume event
    DashObs -> DashObs : updateIncidentQueue()
    DashObs --> Analyst : WebSocket push (new incident)

    MQ -> AuditObs : consume event
    AuditObs -> AuditObs : logIncidentCreation()
    AuditObs -> DB : INSERT audit trail
end

== 8. Response ==
IMS --> ECS : incidentId
ECS --> AIS : success
AIS --> Splunk : 202 Accepted

```

@endum1

## UML Sequence — Incident Response Orchestration

```
@startuml SDA-Pro_Sequence_Response
!theme plain
autonumber

title Sequence: Incident Response Orchestration
(Facade, Strategy, Decorator, State, Observer, Adapter)

actor "SOC Analyst" as Analyst
participant "SOC Dashboard (View)" as View
participant "IncidentController" as Ctrl
participant "ResponseOrchestrationService" as ROS
participant "IncidentResponseFacade (Facade)" as Facade
participant "Incident" as Incident
participant "UnderTriageState (State)" as TriageSt
participant "ContainmentState (State)" as ContainSt
participant "ResponseStrategySelector (Strategy)" as StratSel
participant "AggressiveContainmentStrategy (Strategy)" as Strat
participant "ResponseActionFactory (Factory Method)" as Fact
participant "BlockIPAction" as Action
participant "AuditLogDecorator (Decorator)" as Dec1
participant "ApprovalGateDecorator (Decorator)" as Dec2
participant "MetricsDecorator (Decorator)" as Dec3
participant "ResponseActionProxy (Proxy)" as Proxy
participant "PaloAltoAdapter (Adapter)" as PAAd
participant "EventBusPublisher (Singleton + Observer)" as EB
participant "NotificationDispatcher (Observer)" as NotifObs
participant "DashboardUpdater (Observer)" as DashObs

database "PostgreSQL" as DB
queue "RabbitMQ" as MQ

== 1. Analyst Triggers Response ==
Analyst -> View : Click "Execute Response Plan"
View -> Ctrl : POST /incidents/{id}/respond
Ctrl -> ROS : assessAndRespond(incidentId)

== 2. Facade Coordinates ==
ROS -> Facade : assessAndRespond(incidentId)
Facade -> DB : load incident
DB --> Facade : Incident

== 3. State Transition — Triage to Containment ==
Facade -> Incident : getCurrentState()
Incident --> Facade : UnderTriageState
Facade -> TriageSt : initiateContainment(incident, actions)
TriageSt -> TriageSt : validateAnalystAuthorization()
TriageSt -> Incident : setState(ContainmentState)
Incident -> ContainSt : onEnter(incident)
ContainSt -> ContainSt : logStateTransition()
```



```

== 4. Strategy Selection ==
Facade -> StratSel : selectStrategy(incident)
StratSel -> StratSel : evaluateSeverity(CRITICAL)
StratSel -> StratSel : evaluateAssetCriticality(HIGH)
StratSel --> Facade : AggressiveContainmentStrategy

== 5. Strategy Determines Actions ==
Facade -> Strat : determineActions(incident, context)
Strat -> Strat : analyzeAttackVector()
Strat -> Strat : recommendIsolation()
Strat --> Facade : List<ResponseAction> [BlockIP, IsolateEndpoint]

== 6. Factory + Decorator Pipeline ==
loop For each action in plan

    Facade -> Fact : createAction(BLOCK_IP)
    Fact -> Action : new BlockIPAction(ip, duration)
    Fact --> Facade : action

    Facade -> Dec1 : new AuditLogDecorator(action)
    Facade -> Dec2 : new ApprovalGateDecorator(dec1)
    Facade -> Dec3 : new MetricsDecorator(dec2)
    Facade --> Facade : decoratedAction = dec3

== 7. Decorator Execution Chain ==
Facade -> Dec3 : execute(target)
activate Dec3

Dec3 -> Dec3 : recordMetrics()
Dec3 -> Dec2 : execute(target)
activate Dec2

Dec2 -> Dec2 : checkApprovalRequired()
Dec2 -> Dec2 : approvalGranted = true
Dec2 -> Dec1 : execute(target)
activate Dec1

Dec1 -> Dec1 : preActionAuditLog()
Dec1 -> Action : execute(target)
activate Action

Action -> Proxy : blockIP(ip, duration)
Proxy -> Proxy : validatePermissions()
Proxy -> PAAAd : sendBlockCommand(ip, duration)
PAAAd -> PAAAd : mapToPaloAltoFormat()
PAAAd --> Proxy : success
Proxy --> Action : ActionOutcome.SUCCESS

Action --> Dec1 : outcome
deactivate Action

Dec1 -> Dec1 : postActionAuditLog()
Dec1 --> Dec2 : outcome
deactivate Dec1

Dec2 --> Dec3 : outcome
deactivate Dec2

Dec3 -> Dec3 : recordCompletionMetrics()
Dec3 --> Facade : outcome
deactivate Dec3

Facade -> Incident : recordActionOutcome(outcome)
end

== 8. State Check - Auto-Transition? ==
Facade -> ContainSt : evaluateAutoTransition(incident)
ContainSt -> ContainSt : allContainmentActionsSuccessful()
ContainSt -> Incident : setState(EradicationState)
Incident -> DB : persist()

```

```
== 9. Observer - Event Publication ==
Facade -> EB : publishResponseActionExecuted(incident, outcomes)
EB -> MQ : push "ResponseActionExecuted" event

par Parallel Observer Reactions
  MQ -> DashObs : consume event
  DashObs -> View : WebSocket push
  View --> Analyst : Show "Containment Complete"

  MQ -> NotifObs : consume event
  NotifObs -> NotifObs : evaluateEscalation()
  NotifObs -> NotifObs : sendPagerDutyAlert()
end

== 10. Response Chain ==
Facade --> ROS : ResponsePlanResult
ROS --> Ctrl : ResponseDTO
Ctrl --> View : 200 OK + updated incident
View --> Analyst : Display success + next steps

@enduml
```

